OpenOCD

Overview

OpenOCD, short for Open On-Chip Debugger, is an open-source tool that provides low-level debug access to embedded targets. It sits between your debugger and the hardware, translating high-level debug commands into JTAG or SWD transactions that the target device understands.

In a Zephyr workflow, OpenOCD does not understand your application logic or symbols. Instead, it manages the physical connection to the device, controls the CPU cores, and exposes a GDB server that tools like GDB or VS Code can connect to.

On the ESP32-S3, OpenOCD communicates directly with the built-in USB JTAG interface. This allows flashing, halting, stepping, and inspecting memory using a single USB cable, without the need for an external debug probe. OpenOCD remains running throughout the debug session, acting as the persistent link between your development tools and the hardware.

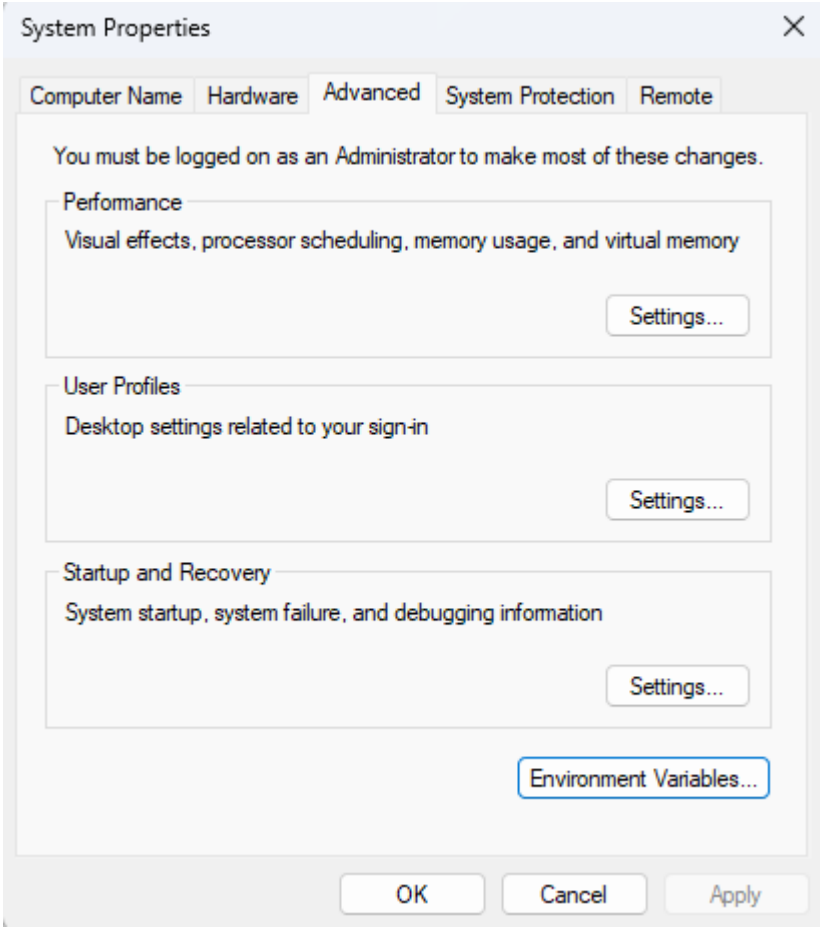
Installing OpenOCD

- OpenOCD Binaries

Copy the unzipped folder to Program Files

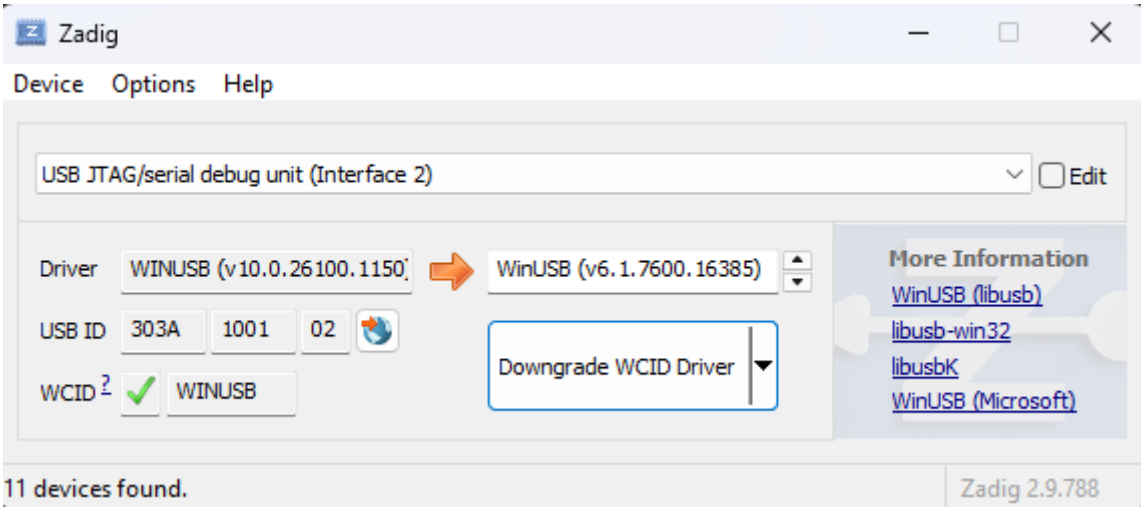
```
C:\Program Files\xpack-openocd-0.12.0-7
```

Add this to our path

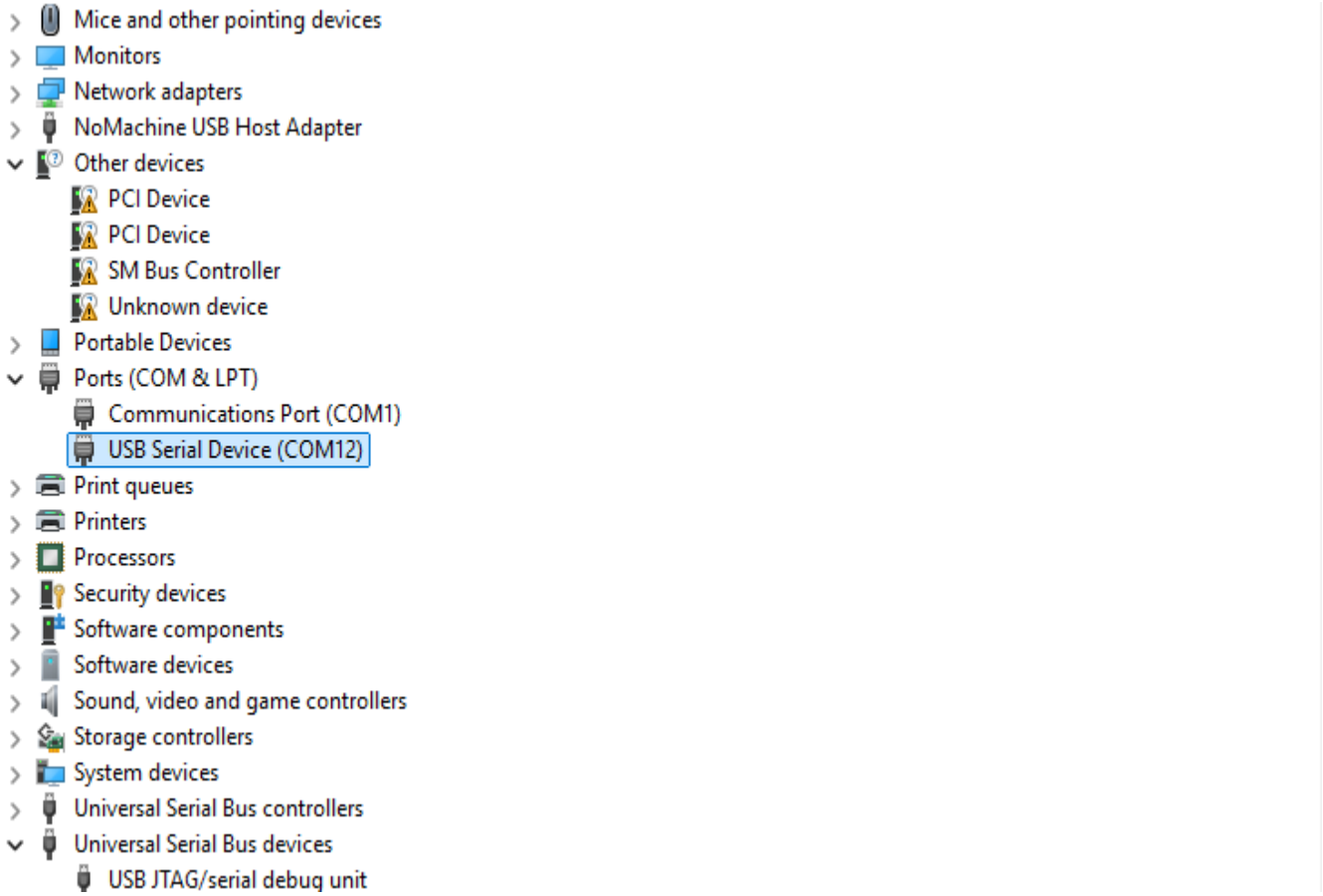


Go to zadig

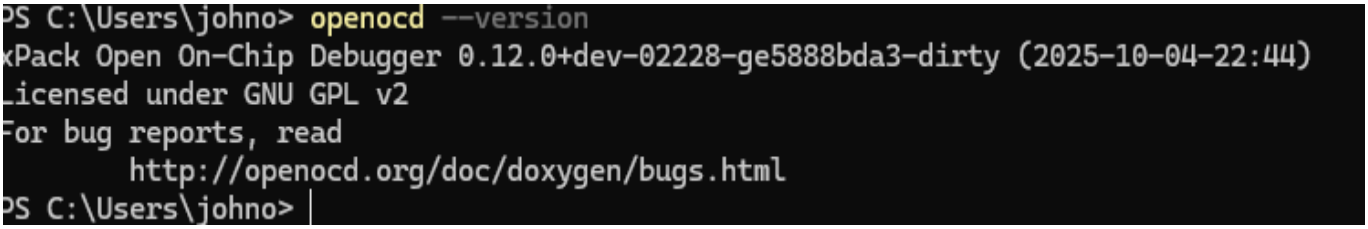
Select the option for USB JTAG/serial debug uint (interface 2) - you will need to downgrade to WinUSB V6.1.xxxxx



After install we should see a new serial port under com ports and under Universal Serial Bus Devices we should see a JTAG option.



Check that we have no errors



Then try and run the service (if you get a port conflict then change the port)

```
openocd -f board/esp32s3-builtin.cfg -c "tcl port 6667"
```

You should see

```
xPack Open On-Chip Debugger 0.12.0+dev-02228-ge5888bda3-dirty (2025-10-04-22:44)
Licensed under GNU GPL v2
For bug reports, read
        http://openocd.org/doc/doxygen/bugs.html
Info : esp_usb_jtag: VID set to 0x303a and PID to 0x1001
Info : esp_usb_jtag: capabilities descriptor set to 0x2000
Info : [esp32s3.cpu0] Hardware thread awareness created
Info : [esp32s3.cpu1] Hardware thread awareness created
force hard breakpoints
adapter speed: 40000 kHz
Info : Listening on port 6667 for tcl connections
```

```
Info : Listening on port 4444 for telnet connections
Info : esp_usb_jtag: Device found. Base speed 40000KHz, div range 1 to 255
Info : clock speed 40000 kHz
Warn : DEPRECATED: auto-selecting transport "jtag". Use 'transport select jtag' to
suppress this message.
Info : JTAG tap: esp32s3.cpu0 tap/device found: 0x120034e5 (mfg: 0x272
(Tensilica), part: 0x2003, ver: 0x1)
Info : JTAG tap: esp32s3.cpu1 tap/device found: 0x120034e5 (mfg: 0x272
(Tensilica), part: 0x2003, ver: 0x1)
Info : [esp32s3.cpu0] Examination succeed
Info : [esp32s3.cpu1] Examination succeed
Info : [esp32s3.cpu0] starting gdb server on 3333
Info : Listening on port 3333 for gdb connections
Info : [esp32s3.cpu0] Debug controller was reset.
Info : [esp32s3.cpu0] Core was reset.
Info : [esp32s3.cpu1] Debug controller was reset.
Info : [esp32s3.cpu1] Core was reset.
```

I had installed my SDK here

```
c:/users/johno/zephyr-sdk-0.17.4/xtensa-espressif_esp32s3_zephyr-elf
```

My gdb is located here

```
c:/users/johno/zephyr-sdk-0.17.4/xtensa-espressif_esp32s3_zephyr-elf/bin/xtensa-
espressif_esp32s3_zephyr-elf-gdb.exe
```

We can invoke with

```
c:/users/johno/zephyr-sdk-0.17.4/xtensa-espressif_esp32s3_zephyr-elf/bin/xtensa-
espressif_esp32s3_zephyr-elf-gdb.exe
.\build\esp32s3_demo_advanced\zephyr\zephyr.elf
```

Then in the gdb console

```
target extended-remote localhost:3333
```

We should see something like

```
(gdb) target extended-remote localhost:3333
Remote debugging using localhost:3333
warning: multi-threaded target stopped without sending a thread-id, using first
non-exited thread
```

```
0x40376f28 in arch_cpu_idle () at
D:/ESP32Zephyr/zephyrproject/zephyr/arch/xtensa/core/cpu_idle.c:14
14      }
```

And in the OCD console

```
Info : accepting 'gdb' connection on tcp/3333
Info : Set GDB target to 'esp32s3.cpu0'
Info : New GDB Connection: 1, Target esp32s3.cpu0, state: halted
```

We can monitor serial output and debug with just a single cable. The cable should be plugged into the JTAG port and we can than adjust our device tree overlay to direct serial output to USB Serial/JTAG by adding the section below:

```
/ {
    chosen {
        zephyr,console = &usb_serial;
        zephyr,shell-uart = &usb_serial;
    };
};

&usb_serial {
    status = "okay";
};
```

Integrating with VSCode

Below is a sample launch.json file for vscode.

```
{
    "configurations": [
        {
            "name": "Debug ESP32-S3",
            "type": "cppdbg",
            "request": "launch",
            "program": "${input:elfPath}",
            "args": [],
            "stopAtEntry": true,
            "cwd":  "${workspaceFolder}",
            "MIMode": "gdb",
            "miDebuggerPath": "c:/users/johno/zephyr-sdk-0.17.4/xtensa-
espressif_esp32s3_zephyr-elf/bin/xtensa-espressif_esp32s3_zephyr-elf-gdb.exe",
            "miDebuggerServerAddress": "localhost:3333",
            "setupCommands": [
                {
```

```
                    "description": "Pretty print for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        }
    ],
    "inputs": [
        {
            "id": "elfPath",
            "type": "promptString",
            "description": "Enter the path to the ELF file",
        }
    ]
}
```